RoadLens - the approach in simple words

A short guide to what the project does, why, and where it stands. No maths needed.

The problem

A self-driving car must read traffic signs. In good weather this is easy: computers already read German traffic signs correctly about 99 times out of 100. But real cameras see rain, fog, snow, darkness, shaking (blur) and grainy pictures (noise). Then the same computer can suddenly get many signs wrong. **RoadLens tries to build a sign reader that stays correct when the picture is damaged.**

The big idea in one line

Teach the model on **slightly messed-up copies** of clean pictures, use **two different kinds of model together**, and test it on **kinds of damage it has never seen**.

A school analogy

• **Textbook** = clean training pictures (GTSRB: about 39,000 photos of German signs, 43 types).

• **Practice questions** = the same pictures, changed a little (rotated, colours changed, shifted, wavy). The model practises on these.

• **Mock exam** = a small set with four special kinds of damage (speckle noise, blur, splashes, strong colour). Used only to decide which saved version of the model is best.

• **Final exam** = the test pictures with 14 kinds of damage at 5 strength levels. **The model never practises on these 14.** That is what makes the test fair.

What the model sees

Every photo is shrunk to 64 x 64 pixels (small, like a real far-away sign). Damage is added *after* shrinking, so it looks the way it would on a small sign.

The steps, one by one

1. Get the data

Download GTSRB (German signs). Keep 10% of the training photos aside for checking. Photos of the same physical sign stay together, so the check is honest (no near-copies on both sides). BTSD (Belgian signs) is used later as a 'different country' test: 22 of its sign types match German ones.

2. Make the 'final exam' (GTSRB-C)

Take all 12,630 test photos and add 14 kinds of damage x 5 strengths = 70 damaged copies of the test set. Made once, with a fixed random seed, so everyone gets the same exam. The strengths are tuned for 64-pixel pictures (see the last pages).

3. Train simple models first (baselines)

SmallCNN: a small network trained from zero. It is the 'ruler': other models are scored relative to it. **ResNet-18** (a picture network) and **DeiT-S** (a transformer, looks at the picture as small patches). Both start from ImageNet knowledge and are fine-tuned. Training data: clean photos + tiny colour changes only. No left-right flipping, because that would turn 'keep left' into 'keep right'.

4. Train with the RoadLens method

For each photo, make 2 'practice' versions by mixing several random changes (AugMix style): contrast stretch, colour levels, rotate, shear, shift, plus 4 newer ones (PRIME style): a wave pattern, a random sharpen/soften filter, a gentle wave-like bend, a random colour curve. Then the model is asked: *give the same answer for the clean photo and both practice versions* (this is the JSD consistency rule).

5. Combine two different models (fusion)

ResNet-18 and DeiT-S make different mistakes. A small 'gate' looks at both and decides, per photo, how much to trust each one. A simple 50/50 average and a 'trust the more confident one' rule are kept as fair comparisons.

6. Adapt during testing (TENT / test-time BN)

While reading a stream of damaged photos, the CNN part adjusts a few internal settings so it becomes more sure of its answers. Tested with groups of 1, 16, 64 and 200 photos.

7. Measure and report

Accuracy, macro-F1 (fair over all 43 sign types), mCE (errors compared with SmallCNN: below 100 = better), ECE (is the model's confidence honest?). Average of 3 runs (seeds 0, 1, 2). Tables in LaTeX, figures as PDF.

Where things stand

What has been run

Run	Status	What it showed
Pilot on Colab T4 (1 seed, short training, 2,000 test photos)	Done, 73 min, no errors	Clean accuracy 98.8-99.6%. ResNet-18 on damaged photos went from 83.6% to 93.6% with the RoadLens training method. The plain 50/50 average (95.4%) slightly beat the learned gate (95.2%). TENT helped only +0.2 points.
Full run (3 seeds, full training, all 12,630 test photos)	Not run yet	Needed for final paper numbers. Rough estimate 10-12 GPU hours on a T4.
v2 improvements (below)	Coded + unit tested, not trained yet	No numbers yet.
Chinese signs (CTSD)	Missing	Needs manual download and a checked class list.

Pilot numbers come from one seed and a small test set, so treat them as a first look, not final results.

The v2 ideas (fixes for what the pilot showed)

• **Self-normalised input**: each photo is brightened/darkened and contrast-adjusted by itself before the model sees it. So fog, dark light or washed-out colours matter less.

• **Extra 'views'**: each model also looks at two smoothed copies of the photo (median filter and soft blur). Smoothing removes grainy noise, which was the weakest point.

• **A smarter gate**: it is trained on held-aside photos the models have not memorised, and it can never end up worse than the plain average (the average is kept as a candidate).

• **Honest confidence**: a single 'temperature' number fixes over- or under-confidence.

• **Safer test-time adaptation**: when only a few photos arrive at once, the old settings are blended in, so adaptation does not break with a group size of 1.

Warning: v2 was designed after looking at the pilot's per-damage results, so GTSRB-C is no longer a fully blind test for v2. The Belgian set (BTSD) is the cleaner check.

What you should do next

• Run experiment 7 (v2) on Colab at pilot scale and compare its rows with the v1 rows.

• If v2 helps, run the full 3-seed experiments.

• Check the corruption strengths on the next pages first: if you change them, the exam changes and everything must be re-run.

Corruption strengths - please check these

These are the exact settings used to damage the test photos (file: roadlens/data/corruptions.py). **Made stronger on 15 Oct 2026.** Noise uses the original big-image benchmark (ImageNet-C) values. Blur, zoom, pixelate, JPEG, fog, frost and elastic use RoadLens's own 64-pixel ladders: S1 clearly visible, S5 about as hard as the original benchmark, evenly harder in between. Snow, brightness and contrast were already strong and stay the same. The older Tiny-ImageNet-C settings were too mild.

Damage	Group	What the number means	S1	S2	S3	S4	S5
Gaussian noise	noise	grain strength (0-1 scale)	0.08	0.12	0.18	0.26	0.38
Shot noise	noise	light amount (LOWER = more grain)	60	25	12	5	3
Impulse noise	noise	share of pixels made black/white	3%	6%	9%	17%	27%
Motion blur	blur	smear size in pixels (sigma)	1.2	2	3.2	4.8	6
Zoom blur	blur	max zoom	1.16x	1.24x	1.32x	1.40x	1.50x
Defocus blur	blur	blur circle radius (px)	1	1.5	2	3	4
Snow	weather	amount of snowflakes (mean)	0.1	0.1	0.15	0.25	0.3
Fog	weather	fog thickness	0.7	1.1	1.5	2.2	3.0
Frost	weather	frost layer weight	0.35	0.5	0.6	0.7	0.8
Brightness	weather	light added (0-1 scale)	0.1	0.2	0.3	0.4	0.5
Contrast	digital	contrast kept (LOWER = flatter)	40%	30%	20%	10%	5%
Pixelate	digital	size kept before blocky upscaling	65%	52%	43%	36%	30%
Elastic	digital	average wobble in pixels	0.75 px	1.5 px	2.25 px	3 px	4 px
JPEG	digital	JPEG quality (LOWER = worse)	45	30	20	12	8

Validation-only damage (the mock exam, never in the final exam): speckle noise 0.15-0.35, Gaussian blur 0.5-1.5 px, spatter (water then mud), saturate (colour x0.3 ... x30).

Quick check: does each level really get harder?

PSNR measures how close the damaged photo is to the clean one (in dB). **Lower = more damage.** For a correct ladder the number should go **down** from S1 to S5. Measured on 150 test photos. 'Package' = the original library settings, shown so you can see how much harsher they were.

Damage	S1	S2	S3	S4	S5	Goes down?	Package S1 / S5
gaussian noise	22.4	19.2	16.1	13.4	10.8	yes	22.4 / 10.8
shot noise	24.1	20.4	17.3	13.8	11.8	yes	24.1 / 11.8
impulse noise	19.8	16.8	15.0	12.3	10.2	yes	19.8 / 10.3
motion blur	33.3	29.3	26.2	24.2	23.2	yes	26.5 / 20.1
zoom blur	27.4	24.8	23.6	22.4	21.8	yes	28.4 / 23.6
defocus blur	40.2	36.7	34.5	30.0	27.9	yes	30.3 / 22.8
snow	17.5	16.3	13.2	11.2	10.2	yes	15.7 / 8.0
fog	19.9	18.1	17.2	15.9	15.3	yes	17.1 / 15.3
frost	13.5	11.0	10.0	9.0	8.6	yes	12.2 / 9.1
brightness	21.7	15.6	12.2	9.8	8.1	yes	21.7 / 8.1
contrast	22.6	21.2	20.1	19.0	18.6	yes	22.6 / 18.6
pixelate	34.1	32.8	30.6	29.6	28.2	yes	33.5 / 27.2
elastic transform	28.8	25.2	23.3	21.9	20.8	yes	27.1 / 21.8
jpeg compression	33.5	32.0	30.6	28.5	26.9	yes	31.5 / 26.4

Notes: all 14 rows now go down steadily. Noise at S4-S5 can make a dark photo almost unreadable; this is the same strength as the standard big-image benchmark. These are no longer the published Tiny-ImageNet-C settings, so say so in the paper. The 10 Oct pilot used the old, milder settings: its numbers cannot be compared with new runs.

Checklist for you

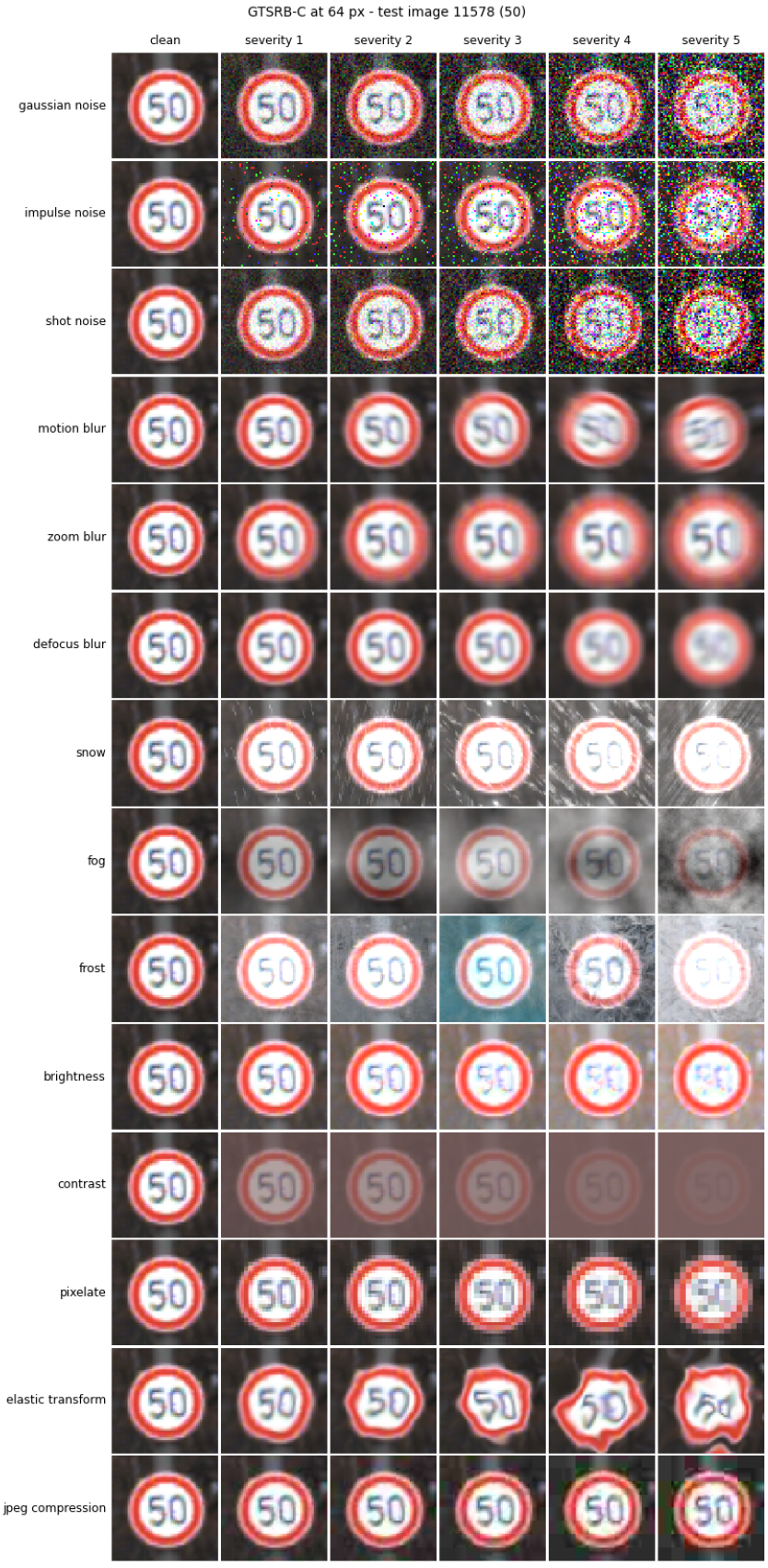
• Does every row get worse from S1 to S5 when you look at the pictures?

• Is S5 still sometimes readable by a human (not pure grey or pure noise)? Contrast S4-S5 is almost flat - this matches the standard benchmark.

• Are any of them now too harsh? Changing them again means re-generating the exam and re-running everything, so decide before the full run.

Look at them yourself

One test sign with every damage at every level (left = clean, then S1 ... S5). A good ladder: S1 is a small change, S5 is hard but a person can often still guess the sign.



Original package settings vs. RoadLens settings

Left three columns: the library's original settings at S1, S3, S5. Right three: the RoadLens settings actually used. The library's blur and elastic still destroy the sign by S3; the RoadLens ladders are strong but keep it recognisable longer.

